

A Summary of Appendix

This appendix complements the main paper by: (i) formalizing notation and filling in omitted derivations; (ii) detailing implementation choices for exact reproduction; (iii) reporting extended results: budget sweeps (Table 7), global vs. local allocation (Table 9), architectural sensitivity (Table 8), calibration-size sensitivity (Table 10), and cross-architecture generalization (Table 12); (iv) specifying the evaluation protocol; and (v) describing export/serving modes and on-device benchmarking.

- [B] Methodological Details and Notation.
- [C] Implementation and Reproducibility.
- [D] Evaluation Protocol.
- [E] Extended Experimental Results.
- [F] Edge Device Deployment.
- [G] Limitations and Future Work.
- [H] LLM Usage Disclosure.

B Methodological Details and Notation

Notation Table. Table 6 summarizes the key mathematical symbols used consistently throughout the paper.

Table 6: Summary of notation.

Symbol	Meaning
L	Total number of layers in the model.
W_l	Weight matrix of layer l with m_l rows and n_l columns.
$\mathbb{R}^{m_l \times n_l}$	
$w_{l,i,j}$	Scalar entry in the i -th row and j -th column of W_l .
L_ℓ, R_ℓ	Low-rank factors such that $\Delta_\ell = L_\ell R_\ell^\top$.
r_ℓ	Selected rank for module ℓ .
R_l	Quantization residual matrix for layer l ($W_l - W_{q,l}$).
r_ℓ	Selected rank (number of protected singular components) for module ℓ .
$g_{\ell,b}(t)$	Gain curve at bit-width b : the error reduction from allocating the t -th singular component (Eq. 18).
ρ	Action density $\rho = \Delta e / \Delta c$ used by the global allocator (Sec. 3.5).

Signal Interpretation. The three signals provide complementary views on parameter importance. Functional Criticality ($g_{l,t}$) measures the first-order sensitivity of the model’s output (with respect to a teacher) to changes along a specific singular direction. It prioritizes directions that are critical for task performance. Optimization Sensitivity ($h_{l,t}$) uses a diagonal Hessian approximation to estimate second-order sensitivity. It identifies directions where small perturbations lead to a large increase in the loss, indicating fragility. Structural Importance ($s_{l,t}$) reflects the magnitude of a singular value, corresponding to its contribution to the Frobenius norm of the weight matrix. It prioritizes directions that are structurally dominant. Fusing these signals allows BALANCER to balance between task-specific importance, optimization stability, and raw reconstruction error, leading to a stronger allocation than any single signal could provide.

C Implementation and Reproducibility

Hyperparameter configuration. Unless otherwise stated, all experiments used a consistent set of configurations. Signals are combined by summation followed by min-max normalization (Sec. 3.3), without introducing tunable fusion weights. Backbone weights were quantized to 4-bit ($b_l=4$), with low-rank factors also in 4-bit precision ($b_r=4$). Per-row scales were stored in 16-bit precision ($b_s=16$). The curvature proxy was estimated with an exponential moving average using momentum $\beta=0.9$. Layers could be assigned zero rank, allowing sparse allocations where corrections were unnecessary.

Computational overhead. Our pipeline naturally decomposes into two phases. (*Phase A*) *Signal profiling* extracts the required signal matrices (teacher-guided gradients, an EMA second-moment/curvature proxy, and activation statistics) and can be cached once per model. (*Phase B*) *Compression given signals* performs two-subspace matrices construction, subspace matrix-induced backbone quantization, subspace matrix-residual spectrum analysis (SVD), global allocation, and weight reconstruction/export. Given precomputed signal matrices, Phase B completes in about 20–30 minutes for LLaMA-3.1-8B on a single A100 (80GB). We report Phase A runtime separately since it depends on the teacher choice, calibration size, and profiling frequency.

Environment and code availability. All experiments were conducted on NVIDIA A100 (80GB) GPUs with CUDA 12.1, PyTorch 2.2, and Transformers 4.41. Evaluation scripts and source code will be included in the supplementary materials.

Low-bit stabilization. For layers quantized below 4-bit ($b_l=2$), we found randomized block-orthogonal preconditioning improves stability in both scoring and reconstruction. This step prevents degenerate singular directions that arise under extreme quantization. For deployment backends requiring static scales (e.g., certain TensorRT variants), we additionally support a global per-matrix scale at export. All storage costs reported in this work are based on the more general per-row ledger defined in Sec. 3.1.

D Evaluation Protocol

This section details the evaluation rules, subspace matrix definitions, and environment configuration that underlie all reported results in Sec. 4.

Subspace Matrix Definitions.

- **Bits:** The average bits-per-parameter implied by the global budget ledger in Eq. 3, i.e., total stored bits divided by N . This cost includes storage for quantized integers *plus* all metadata (per-row scales and low-rank factors).
- **CR:** The Compression Ratio relative to a 16-bit (FP16) baseline, calculated as $16/\bar{b}$.
- **Perplexity (PPL):** We use the standard concatenation-and-block protocol with a context length of 2048, as implemented in the Hugging Face Transformers library.

- **Latency (Lat):** First-token latency, measured in milliseconds (ms). This is the time from the start of the request to the generation of the first token, for a prompt length of 512.
- **Throughput (Tok/s):** Steady-state decoding throughput, measured in tokens per second. This is calculated during the generation of the subsequent 512 tokens after the prefill/prompt phase.
- **VRAM (GB):** We report resident model-state memory in gigabytes during steady-state decoding. This includes the model weights, KV cache, and any persistent framework overhead.

Environment and Scripts. All server-side experiments were conducted on NVIDIA A100 (80GB) GPUs using CUDA 12.1, PyTorch 2.2, and Transformers 4.41. On-device experiments were benchmarked on the Jetson Orin Nano (8GB). To ensure reproducibility, we provide all evaluation scripts used to generate the results, including the exact commands for running baselines and calculating subspace matrix.

E Extended Experimental Results

E.1 Datasets and LLMs

Datasets

- **WikiText-2:** A word-level language modeling corpus from verified Wikipedia articles; we use it for perplexity evaluation.
- **C4 (Colossal Clean Crawled Corpus):** A large cleaned web-crawl corpus; we follow the standard setup for perplexity evaluation. [Raffel *et al.*, 2020]

LLMs

- **LLaMA-2-7B:** A 6.7B-parameter decoder-only model; we use the base model for PTQ. [Touvron *et al.*, 2023]
- **LLaMA-3 / 3.1-8B:** We report results on the 8B model from the LLaMA 3 family. [Dubey *et al.*, 2024]
- **Mistral-7B:** A 7B decoder with grouped-query attention and sliding-window attention. [Jiang *et al.*, 2023]

E.2 Compression Budget vs. Perplexity

Table 7 summarizes LLaMA-3.1-8B results across budgets and shows budget-perplexity sweep as the compression budget tightens (4.2→3.0 bits). The fraction of layers receiving any low-rank correction (“Layers w/ Rank”, i.e., layers with nonzero rank) decreases in tandem, indicating controllable behavior under stricter bit constraints.

E.3 Architectural Sensitivity Analysis

Table 8 compares the relative PPL improvement of BALANCER over uniform Q4 on LLaMA-2-7B and LLaMA-3.1-8B. The gains are more pronounced on LLaMA-2-7B, as LLaMA-3.1-8B is inherently more robust to quantization. This demonstrates that BALANCER automatically adapts, allocating less corrective rank when the base model is already strong.

Table 7: BALANCER performance across different average bit budgets on LLaMA-3.1-8B.

Target Bits	Bits	PPL-W↓	PPL-C4↓	% Layers w/ Rank↓
4.2	4.20	6.698	10.534	53.1
4.0	4.00	6.721	10.583	46.9
3.8	3.80	6.845	10.824	42.2
3.6	3.60	6.781	10.692	37.5
3.2	3.20	7.120	10.980	31.3
3.0	3.08	7.315	11.150	28.1

Table 8: Cross-architectural performance comparison at a 4.0-bit budget (WikiText-2 PPL).

Architecture	FP16	Q4 (RTN)	BALANCER	Δ vs. RTN
LLaMA-3.1-8B	6.627	9.918	6.721	3.197
LLaMA-2-7B	5.470	7.051	5.950	1.101

E.4 Global vs. Local Allocation

A core principle of BALANCER is its global allocation strategy. To rigorously validate this design choice, we compare it against a strong “local” baseline.

Defining the Local Allocator. The local allocator distributes the total low-rank bit budget (the portion of the budget B not used by the quantized backbone) among layers proportional to each layer’s parameter count. Each layer then runs an independent greedy selection using its local budget. Unlike the global allocator, the local version cannot transfer budget from strong layers to more fragile ones.

Table 9: Global vs. local allocation on LLaMA-3.1-8B. The global allocator (BALANCER) yields lower perplexity for the same budget.

Allocator	Bits	PPL-W↓	PPL-C4↓
Local (per-layer)	3.6	6.901	10.872
Global (BALANCER)	3.6	6.781	10.692
Local (per-layer)	3.2	7.330	11.210
Global (BALANCER)	3.2	7.120	10.980

The performance gap substantiates the importance of global cost optimization. The global allocator can divert bits from less sensitive layers to protect disproportionately fragile regions (e.g., early and late transformer blocks). In contrast, the local allocator may waste local budget on low-impact directions while fragile layers may lack sufficient local budget to protect all critical directions.

E.5 Calibration Size Sensitivity

Table 10 shows that performance is stable across 16–128 calibration samples at $B=4.0$ on LLaMA-3.1-8B. Increasing the size from 16 to 64 yields small gains, while beyond 64 changes are negligible. We therefore adopt 64 samples by default.

Table 10: Calibration-size sensitivity on LLaMA-3.1-8B at $B=4.0$. Overlap is the Jaccard index of the top-64 selected directions relative to a 128-sample reference.

Samples	PPL-W↓	PPL-C4↓	Overlap↑
16	6.753	10.621	0.72
32	6.735	10.599	0.86
64	6.721	10.583	0.92
128	6.721	10.582	1.00

E.6 Preprocessing Cost and Resources

Table 11 reports the one-time preprocessing cost of BALANCER on LLaMA-3.1-8B. The full pipeline completes in about 2.3 hours on a single A100 80GB, with peak memory use below 23 GB and total disk footprint under 10 GB. Since profiling is performed once per model and cached, its cost is amortized over the checkpoint lifetime in deployment.

Table 11: Preprocessing cost of BALANCER on LLaMA-3.1-8B (single A100 80GB). “Time” is wall-clock per stage.

Stage	Time (min)	Peak VRAM (GB)	GPU Hours	Disk (GB)
Gradient extraction	45.2	22.5	0.75	2.4
Curvature estimation	32.6	20.8	0.54	1.8
Spectral computation	18.4	19.6	0.31	0.9
Signal fusion	8.2	8.4	0.14	0.3
Global allocation	12.8	6.2	0.21	0.1
Weight reconstruction	22.4	21.2	0.37	3.6
Total	139.6	22.5	2.32	9.1

E.7 Cross-architecture Applicability and Downstream Performance

Generalization to Mistral-7B. To demonstrate that BALANCER is not limited to a single model family, we applied it to Mistral-7B. As shown in Table 12, BALANCER maintains its advantage over uniform quantization at a matched 4.0-bit budget.

Table 12: Generalization results on Mistral-7B. Perplexity (PPL) on WikiText-2.

Method	Bits	PPL-W↓
FP16	16.0	5.120
Q4 (RTN)	4.0	5.580
BALANCER (Q4+LR)	4.0	5.350

E.8 Signal Fusion Ablation

We ablate the construction in Eq. 10–Eq. 11 by enabling different subsets of signals while keeping the same allocator and budget ledger. All settings use the same calibration protocol as Sec. 4.1.

The results show clear complementarity among signals: any single proxy is consistently worse than partial fusions, and the full three-signal construction achieves the lowest

Table 13: Signal-fusion ablation for Eq. 10–Eq. 11 on LLaMA-3.1-8B at $B=4.0$. All variants use the same allocator, budget ledger, calibration protocol, and reconstruction pipeline; only the enabled signals differ. Lower is better.

Subspace signals used	PPL-W↓
Activation only ($\sqrt{c}$)	6.892
Gradient only ($g^{(x)}, g^{(y)}$)	6.945
Curvature only (h)	7.031
Activation + Gradient	6.784
Activation + Curvature	6.812
Gradient + Curvature	6.876
All (Activation + Gradient + Curvature)	6.721

PPL. The best two-signal variants approach the full model but remain inferior, suggesting that gradient and curvature provide non-redundant cues beyond activation statistics. These findings support our conservative fusion design in Eq. 10–Eq. 11: bounded normalization keeps each cue on a comparable scale, and summation provides a robust, tuning-free union of importance that avoids over-committing to a single proxy.

F Export and Deployment

F.1 Export Modes

A practical requirement of post-training compression is that the resulting weights remain compatible with standard inference engines. BALANCER supports two equivalent export modes, both relying solely on standard GEMM kernels at run-time. *Folded export* applies low-rank corrections offline and re-quantizes to obtain a single set of integer weights, incurring no overhead relative to uniform PTQ. *Pack-at-load export* stores the quantized backbone and low-rank factors separately in a compact format; at load time, a one-time reconstruction produces the same runtime layout as folded export. Both modes yield identical execution behavior and are fully compatible with PyTorch/Transformers.

F.2 Deployment Results

Table 14 reports deployment statistics for LLaMA-2-7B at a 4.0-bit budget. Both export modes match the quantized model’s accuracy, VRAM, and throughput.

G Limitations and Future Work

While BALANCER shows strong gains across multiple models and hardware targets, several limitations remain. First, our study is limited to models up to 8B parameters; scaling the allocator to 30B–70B models may require additional engineering to keep profiling and allocation costs tractable. Second, we restrict all evaluations to post-training compression without task-specific fine-tuning. How rank allocation interacts with lightweight adaptation methods such as LoRA or adapters remains unexplored. Third, our experiments primarily focus on perplexity and a small set of reasoning tasks; broader downstream evaluations (e.g., instruction following, multilingual and domain-specific benchmarks) are

Table 14: Deployment results for LLaMA-2-7B at a 4.0-bit budget. Both export modes are fully compatible with standard inference engines.

Mode	PPL-W↓	VRAM (GB)↓	Load Time (s)↓	Storage (GB)↓	Tok/s↑
FP16 baseline	5.470	13.1	8.4	13.1	42.9
Fold export	5.950	3.4	12.6	3.6	43.0
Pack-at-load	5.950	3.4	6.8	2.1	42.5

necessary to fully assess robustness. Finally, although our exported models are compatible with standard inference stacks, deployment across heterogeneous mobile and edge runtimes may expose kernel-level inefficiencies that are not captured in our current experiments.

Future work can address these aspects by extending BALANCER to larger models, integrating with adaptation pipelines, and conducting more comprehensive downstream assessments. We also see opportunities to co-design BALANCER with compiler- and runtime-level optimizations, where budget-aware allocation could complement scheduling, memory management, and hardware specialization.

LLM Usage Disclosure

Large Language Models (LLMs) were used solely as general-purpose assist tools for writing support (e.g., copy-editing, grammar suggestions) and L^AT_EX formatting. LLMs were not used for ideation of the core technical approach, experimental design, data generation, or analysis. All algorithms, experiments, and conclusions were conceived, implemented, and verified by the authors. The authors take full responsibility for the content of this paper, including any text that benefited from LLM-assisted editing.